

# Structure-Aware Graph Contrastive Learning via Degree-Preserving Rewiring

Emely Weigel<sup>[0009-0004-3244-9141]</sup>, Jan von Pichowski<sup>[0009-0001-1541-6477]</sup>, and  
Ingo Scholtes<sup>[0000-0003-2253-0216]</sup>

Center for Artificial Intelligence and Data Science,  
Julius-Maximilians-Universität Würzburg, Würzburg, Germany  
`{firstname.surname}@uni-wuerzburg.de`

**Abstract.** Contrastive learning has emerged as a powerful paradigm for self-supervised graph representation learning. However, its effectiveness strongly depends on the choice of graph augmentations. Existing methods largely rely on random perturbations, which can distort key characteristics such as degree distributions and community structure. In this work, we propose a principled augmentation strategy based on degree-preserving edge rewiring, inspired by the configuration model. This approach generates alternative graph views that maintain node degrees while randomizing the graph topology. We evaluate our method across five datasets and compare it against random augmentations and classical baselines. Our results show that degree-preserving augmentation yields competitive or improved performance, highlighting its potential for graph contrastive learning. This is a work-in-progress contribution.

**Keywords:** graph contrastive learning · graph augmentation · configuration model

## 1 Introduction

Graph Neural Networks [12, 27, 9, 30] have become the standard tool for learning on graph-structured data, but their effectiveness typically depends on large amounts of labeled data. Self-supervised contrastive learning [3] addresses this by training encoders without labels, pulling together two augmented views of the same graph while pushing apart views of different graphs. Methods such as GraphCL [33], GRACE [36], and BGRL [24] have shown that the resulting representations can match or approach fully supervised baselines.

The key design decision in a contrastive method is the augmentation strategy. Good augmentations must introduce enough variation to force non-trivial learning, yet preserve the structural patterns that make two views of the same graph meaningfully similar [25]. Standard approaches, like random edge dropping, node removal or attribute masking, treat all edges as equally expendable and offer no guarantee about which structural properties survive. This is problematic: random edge dropping disproportionately distorts high-degree hub nodes and systematically degrades dense community structure, causing the model to learn invariances over augmentation artifacts rather than genuine graph properties [17].

We propose *degree-preserving edge rewiring*: augmented views are sampled from the configuration model [16, 7], a null model, which randomises the topology of a graph while preserving the degree sequence. This gives the contrastive objective a clear semantic: the encoder learns to be invariant to random rewiring. This paper makes the following contributions:

- We identify degree-dependent distortions and community structure bias as failure modes of random augmentations in graph contrastive learning.
- We propose degree-preserving edge rewiring as a principled augmentation strategy grounded in the configuration model.
- We evaluate our method in five benchmark datasets with four GNN encoders, showing consistent training loss improvements, better-structured embeddings, and competitive downstream accuracy.

## 2 Related Work

*Graph self-supervised and contrastive learning.* Self-supervised learning on graphs has been driven by mutual information maximisation and contrastive objectives. Deep Graph Infomax [26] maximises mutual information between node and graph-level representations using feature-shuffling. GraphCL [33] extends this to a multi-view InfoNCE framework with randomly augmented graph pairs. GRACE [36] and MVGRL [10] develop node-level and multi-scale contrastive objectives, while bootstrapping approaches such as BGRL [24] remove the need for explicit negative pairs. Non-contrastive methods, including Node2Vec [8] and Laplacian positional encodings [5], provide unsupervised baselines that require no labels. Our method builds on the GraphCL framework, using these approaches as downstream-evaluation baselines.

*Augmentation strategies.* The effectiveness of graph contrastive learning is highly sensitive to the choice of augmentation strategies. GraphCL [33] evaluates node dropping, edge perturbation, attribute masking, and subgraph sampling, finding that no single strategy dominates across datasets. Adaptive augmentation [37] adjusts perturbation rates using node centrality; automated selection [32] searches over strategies without labelled data. Adversarial augmentation [23] generates harder views by maximising the contrastive loss. SimGRACE [29] avoids graph-level perturbation entirely by perturbing encoder weights instead. Despite this diversity, all these methods apply random structural perturbations that do not control for graph invariants such as degree distributions, a limitation we formalise and address in our work.

*Structure-aware and principled augmentations.* Tian et al. [25] show that contrastive views should preserve task-relevant information, motivating principled augmentation design. COSTA [35] performs augmentation in the spectral feature space to reduce structural distortion. Bao et al. [1] introduce co-regularised augmentations that align views at multiple granularities. These works argue

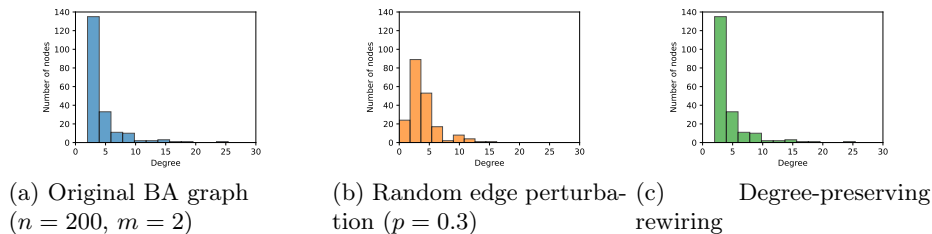


Fig. 1: Degree distributions of a Barabási-Albert (BA) graph before (a) and after random edge perturbation (b) and degree-preserving edge rewiring (c).

for structure-guided perturbations, yet none explicitly constrains the degree sequence. Our method provides exactly this guarantee: every augmented view is drawn from the configuration model [16, 7], a null model for degree-sequence-preserving random graphs, connecting graph contrastive learning to a well-established framework in network science [19].

### 3 Method

We first formalise two structural failure modes of random augmentations: degree-dependent noise and biased community disruption (3.1). We then propose *degree-preserving edge rewiring*, grounded in the configuration model [16, 7], as a principled solution that preserves the degree sequence while randomising a graph’s topology (3.2–3.3). This augmentation is embedded into the GraphCL [33] pipeline, replacing random edge perturbation (3.4).

#### 3.1 Limits of Random Augmentations

Random edge augmentation strategies, like edge dropping, are standard in graph contrastive learning [33, 37, 32, 23], but can cause structural distortions that are problematic for learning high-quality node embeddings [28].

*Degree Distribution Distortion.* When edges are dropped independently with probability  $p$ , the degree of node  $i$  in the perturbed graph  $\tilde{G}$  follows a binomial distribution:  $\mathbb{E}[\tilde{d}_i] = (1 - p) \cdot d_i$ ,  $\text{Var}(\tilde{d}_i) = d_i \cdot p(1 - p)$ . Absolute fluctuations in terms of the standard deviation scale with  $\sqrt{d_i}$ , so high-degree hub nodes are disproportionately affected. A model trained to be invariant to this perturbation is implicitly trained to discount node degrees, which however is a highly informative characteristic in real-world networks [17, 34]. Figure 1 illustrates this.

*Community Structure Disruption.* Dense communities contain more edges and therefore lose more connections in expectation under random edge dropping, introducing a systematic bias: in networks with high modularity [18], densely connected communities are weakened internally while sparse inter-community

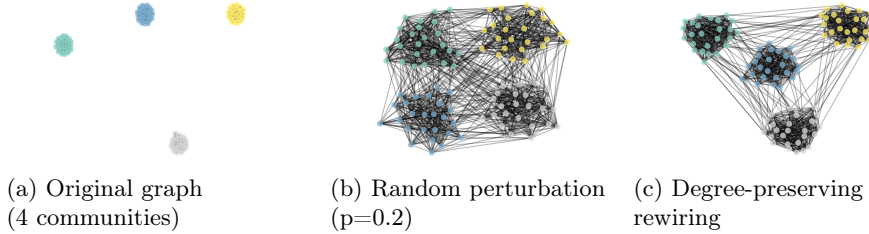


Fig. 2: Visual comparison of community structure preservation. Random perturbation weakens community boundaries, while degree-preserving rewiring better maintains modular structure.

links remain largely intact. Figure 2 illustrates this: random perturbations (b) weaken community boundaries, while degree-preserving rewiring (c) better maintains the modular structure of the graph.

*Semantic Invariance Violation.* Random edge dropping does not guarantee semantic equivalence [25]: excessive removal can give the same node entirely different local neighborhoods across views, turning positive pairs into false positives.

### 3.2 Proposed Solution: Degree-Preserving Perturbations

We propose generating augmented views via degree-preserving edge rewiring, grounded in the configuration model [16, 7]. The configuration model is a null model for random graphs with a prescribed degree sequence [15]: given  $G = (V, E)$  with degree sequence  $(d_1, \dots, d_n)$ , it samples a graph  $G' = (V, E')$  uniformly from all edge sets satisfying  $d'_i = d_i$  for all  $i \in V$ , randomising topological features, such as clustering, path lengths or community membership, while preserving node degrees. To our knowledge, this null model has not previously been used to generate contrastive views. Prior use has been confined to network analysis and hypothesis testing [2, 15, 19, 13, 21].

*Degree-Preserving Edge Rewiring.* We approximate sampling from the configuration model via an MCMC edge-swap procedure [7] (Algorithm 1, illustrated in Figure 3): at each step, two edges are selected uniformly at random and their endpoints swapped if the swap introduces no self-loops or duplicate edges.

*Complexity.* Each swap attempt is  $O(1)$  on an edge list, so  $k$  attempts cost  $O(k)$ . Setting  $k = |E|$ , that is the default in our experiments, gives cost linear in the number of edges. This is negligible relative to the GNN forward pass.

*Choice of  $k$ .* The number of swap attempts  $k$  can be understood as a perturbation-strength parameter, similar in spirit to the drop probability  $p$  in random edge dropping. We choose  $k$  on the order of  $|E|$  rather than a multiple of it, since a very large  $k$  would likely let the MCMC chain approach mixing, at which point  $G'$

would resemble an essentially independent sample from the configuration-model ensemble rather than a controlled perturbation of  $G$ . In that case, the two augmented views might share little beyond the degree sequence, which could weaken the contrastive signal in a way similar to the semantic-invariance concerns discussed for random edge dropping in Section 3.1. Our choice of  $k$  is therefore intended as a moderate perturbation budget: large enough to introduce meaningful topological variation, but small enough to keep the two views recognisably related.

### 3.3 Why DP-Rewire Addresses the Gap

Degree-preserving rewiring directly eliminates the failure modes above. By construction,  $\text{Var}(d'_i) = 0$  for all  $i$  and through the swapping, high-degree nodes in dense communities lose and gain edges at the same rate as low-degree nodes on the network border, which prevents the steady loss of modularity seen with random edge dropping. The main idea is that a node’s degree is an important node characteristic, while the exact topology of connections is less important. Contrastive learning then focuses on the differences that remain, helping the encoder learn embeddings that are stable under random rewiring but still capture the structural properties that genuinely distinguish nodes. This is important because node degrees predict various node characteristics in real-world networks [17].

### 3.4 Contrastive Learning Framework

Following GraphCL [33], we apply degree-preserving edge rewiring twice to obtain two independent views  $G_1, G_2$  of the input graph (Figure 3). Both views are encoded by a shared GCN  $f_\theta$  and projected by a two-layer MLP  $g_\phi$ , yielding  $\ell_2$ -normalised representations  $p_1, p_2$ . The encoder is trained with the bidirectional InfoNCE loss  $\mathcal{L}$  [20]:

$$\mathcal{L}_i = -\log \frac{\exp(p_i^{(1)\top} p_i^{(2)} / \tau)}{\sum_{j=1}^n \exp(p_i^{(1)\top} p_j^{(2)} / \tau)}, \quad \mathcal{L} = \frac{1}{2n} \sum_{i=1}^n (\mathcal{L}_i^{(1 \rightarrow 2)} + \mathcal{L}_i^{(2 \rightarrow 1)})$$

The projection head is discarded after pretraining; downstream evaluation uses the encoder representations  $Z = f_\theta(G)$  directly.

*Interpretability of  $\mathcal{L}$ .* Minimising  $\mathcal{L}$  can be interpreted as encouraging the encoder to learn representations that are *approximately invariant to graph topology given a fixed degree sequence*, which links the contrastive objective conceptually to the configuration model.

## 4 Evaluation

We evaluate on five benchmark datasets summarised in Appendix Table 4: Cora, PubMed [31], and Amazon Computers [22] as homophilic citation and co-purchase graphs, and Roman-Empire and Minesweeper as heterophilic graphs [14].

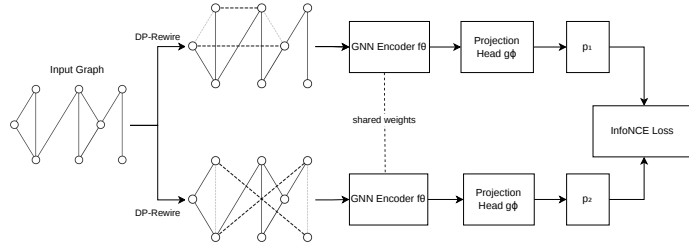


Fig. 3: Overview of the contrastive learning framework. The input graph is augmented twice via degree-preserving edge rewiring to create two views, which are passed through a shared GNN encoder and projection head to produce representations  $p_1$  and  $p_2$  optimised with the InfoNCE loss.

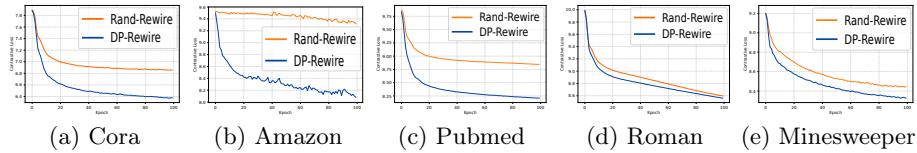


Fig. 4: InfoNCE training loss for **Rand-Rewire** and **DP-Rewire**.

We run two complementary experiments: (i) an edge-rewiring-only setting comparing random rewiring (Rand-Rewire) against our degree-preserving variant (DP-Rewire) to isolate the effect of the augmentation strategy, and (ii) a full GraphCL pipeline (DP-Aug vs. Rand-Aug) with three non-contrastive baselines: raw features, Laplacian PEs [5], and Node2Vec [8]. Both experiments use a GCN [12] encoder with a linear SVM and a three-layer MLP as downstream classifiers.

We follow the standard two-stage SSL protocol [3]: the encoder is pretrained without labels, then frozen and a classifier is trained with labelled nodes.

#### 4.1 Contrastive Training Dynamics

*Training Loss.* Figure 4 shows that DP-Rewire reaches a lower final InfoNCE training loss than Rand-Rewire on every dataset. The effect is most pronounced on the homophilic graphs, where degree-preserving views form more semantically consistent positive pairs. It is smaller but positive on the heterophilic graphs. This lower loss likely reflects that DP-Rewire’s two views are inherently more similar to each other, since much of the topology is preserved by construction, making positive pairs easier to align. This also explains why the loss advantage does not always translate directly into downstream gains (Section 4.2): a lower loss reflects view consistency rather than embedding discriminativeness, and the augmentation still yields competitive or improved accuracy on most datasets despite this distinction.

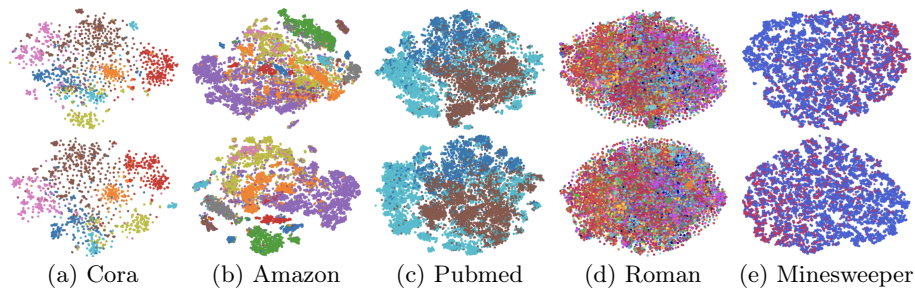


Fig. 5: t-SNE embeddings of node representations learned by Rand-Rewire (top row) and DP-Rewire (bottom row). Colour indicates class membership.

Table 1: Node classification accuracy (Rand-Rewire vs. DP-Rewire) with GCN.

|     | Method       | Cora                                | Amazon                              | PubMed                              | Roman-Empire                        | Minesweeper                         | Avg. Gain |
|-----|--------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-----------|
| MLP | Rand-Rewire  | $0.768 \pm 0.021$                   | $0.911 \pm 0.004$                   | <b><math>0.757 \pm 0.009</math></b> | <b><math>0.959 \pm 0.003</math></b> | <b><math>0.824 \pm 0.001</math></b> | –         |
|     | DP-Rewire    | <b><math>0.794 \pm 0.009</math></b> | <b><math>0.917 \pm 0.005</math></b> | $0.756 \pm 0.017$                   | $0.948 \pm 0.004$                   | $0.823 \pm 0.002$                   | –         |
|     | Rel-Gain (%) | 3.33                                | 0.65                                | −0.13                               | −1.14                               | −1.21                               | 0.30      |
| SVM | Rand-Rewire  | $0.768 \pm 0.018$                   | $0.857 \pm 0.012$                   | $0.769 \pm 0.011$                   | <b><math>0.428 \pm 0.003</math></b> | <b><math>0.629 \pm 0.002</math></b> | –         |
|     | DP-Rewire    | <b><math>0.790 \pm 0.013</math></b> | <b><math>0.887 \pm 0.004</math></b> | <b><math>0.779 \pm 0.005</math></b> | $0.425 \pm 0.003$                   | $0.626 \pm 0.004$                   | –         |
|     | Rel-Gain (%) | 2.86                                | 3.50                                | 1.30                                | −0.70                               | −0.47                               | 1.29      |

*Embedding Structure.* Figure 5 shows t-SNE embeddings. On homophilic datasets DP-Rewire produces markedly better-separated class clusters (note the orange cluster); on heterophilic datasets class boundaries are less pronounced under both strategies, with DP-Rewire achieving comparable separation.

## 4.2 Results

*Degree-preserving vs random rewiring.* Table 1 compares Rand-Rewire and DP-Rewire in the edge-only augmentation setting. DP-Rewire outperforms Rand-Rewire on all three homophilic datasets, with gains of up to +3.5% on Amazon (SVM) and +3.3% on Cora (MLP). On the heterophilic datasets Roman-Empire and Minesweeper, DP-Rewire slightly underperforms, with negative relative gains of up to −1.2%. Average gains are +0.30% for MLP and +1.29% for SVM, with the larger SVM benefit suggesting that better-structured embeddings matter most when the downstream model has limited capacity.

*DP-Aug in a full GraphCL pipeline.* Table 2 reports results for the complete GraphCL pipeline. With the MLP classifier, DP-Aug achieves the best average rank (1.8 vs. 2.0 for Rand-Aug), winning outright on Cora and Roman-Empire. With the SVM classifier, Rand-Aug leads (avg. rank 1.8 vs. 2.4 for DP-Aug), performing best on Cora, PubMed, and Minesweeper. Notably, on Amazon the Laplacian baseline tops the MLP ranking, indicating that rich spectral features can substitute for contrastive pretraining on this densely connected

Table 2: Node classification accuracy (%) with GCN embeddings across datasets.

|     | Method             | Cora                                    | Amazon                                  | PubMed                                  | Roman-Empire                            | Minesweeper                             | Avg. Rank  |
|-----|--------------------|-----------------------------------------|-----------------------------------------|-----------------------------------------|-----------------------------------------|-----------------------------------------|------------|
| MLP | Features           | $0.580 \pm 0.032$ (5)                   | $0.868 \pm 0.005$ (4)                   | $0.734 \pm 0.007$ (3)                   | $0.712 \pm 0.071$ (4)                   | $0.805 \pm 0.002$ (4)                   | 4.0        |
|     | Laplacian          | $0.581 \pm 0.028$ (4)                   | <b><math>0.916 \pm 0.003</math></b> (1) | $0.728 \pm 0.006$ (4)                   | $0.749 \pm 0.026$ (3)                   | $0.804 \pm 0.002$ (5)                   | 3.4        |
|     | Node2Vec           | $0.736 \pm 0.020$ (3)                   | $0.867 \pm 0.003$ (5)                   | $0.709 \pm 0.027$ (5)                   | $0.179 \pm 0.016$ (5)                   | <b><math>0.853 \pm 0.009</math></b> (1) | 3.8        |
|     | Rand-Aug (GraphCL) | $0.816 \pm 0.019$ (2)                   | $0.904 \pm 0.006$ (2)                   | <b><math>0.793 \pm 0.008</math></b> (1) | $0.874 \pm 0.002$ (2)                   | $0.813 \pm 0.001$ (3)                   | <u>2.0</u> |
|     | DP-Aug (ours)      | <b><math>0.820 \pm 0.008</math></b> (1) | $0.895 \pm 0.005$ (3)                   | $0.790 \pm 0.017$ (2)                   | <b><math>0.907 \pm 0.006</math></b> (1) | $0.815 \pm 0.002$ (2)                   | <b>1.8</b> |
| SVM | Features           | $0.603 \pm 0.003$ (4)                   | $0.857 \pm 0.000$ (4)                   | $0.732 \pm 0.003$ (4)                   | <b><math>0.613 \pm 0.001</math></b> (1) | $0.543 \pm 0.001$ (3)                   | 3.2        |
|     | Laplacian          | $0.602 \pm 0.003$ (5)                   | $0.857 \pm 0.000$ (4)                   | $0.732 \pm 0.003$ (4)                   | <b><math>0.613 \pm 0.001</math></b> (1) | $0.543 \pm 0.001$ (3)                   | 3.4        |
|     | Node2Vec           | $0.701 \pm 0.011$ (3)                   | <b><math>0.881 \pm 0.006</math></b> (1) | $0.735 \pm 0.009$ (3)                   | $0.037 \pm 0.003$ (5)                   | $0.530 \pm 0.009$ (5)                   | 3.4        |
|     | Rand-Aug (GraphCL) | <b><math>0.820 \pm 0.015</math></b> (1) | <u><math>0.861 \pm 0.002</math></u> (2) | <b><math>0.801 \pm 0.007</math></b> (1) | $0.406 \pm 0.005$ (4)                   | <b><math>0.621 \pm 0.003</math></b> (1) | <b>1.8</b> |
|     | DP-Aug (ours)      | <u><math>0.812 \pm 0.024</math></u> (2) | $0.859 \pm 0.004$ (3)                   | <u><math>0.799 \pm 0.011</math></u> (2) | $0.411 \pm 0.005$ (3)                   | <u><math>0.619 \pm 0.003</math></u> (2) | <u>2.4</u> |

Table 3: Node classification accuracy (%) with DP-Aug across datasets.

|     | Method     | Cora                                    | Amazon                                  | PubMed                                  | Roman-Empire                            | Minesweeper                             | Avg. Rank  |
|-----|------------|-----------------------------------------|-----------------------------------------|-----------------------------------------|-----------------------------------------|-----------------------------------------|------------|
| MLP | SAGE       | $0.763 \pm 0.007$ (4)                   | $0.878 \pm 0.006$ (3)                   | $0.767 \pm 0.012$ (3)                   | <b><math>0.964 \pm 0.002</math></b> (1) | <b><math>0.871 \pm 0.001</math></b> (1) | 2.4        |
|     | GAT        | <u><math>0.813 \pm 0.008</math></u> (2) | <b><math>0.899 \pm 0.004</math></b> (1) | <b><math>0.790 \pm 0.011</math></b> (1) | $0.843 \pm 0.005$ (3)                   | $0.813 \pm 0.001$ (3)                   | <u>2.0</u> |
|     | GIN        | $0.776 \pm 0.014$ (3)                   | $0.833 \pm 0.006$ (4)                   | $0.765 \pm 0.013$ (4)                   | $0.768 \pm 0.013$ (4)                   | $0.806 \pm 0.001$ (4)                   | 3.8        |
|     | GCN (main) | <b><math>0.820 \pm 0.008</math></b> (1) | <u><math>0.895 \pm 0.005</math></u> (2) | <b><math>0.790 \pm 0.017</math></b> (1) | <u><math>0.907 \pm 0.006</math></u> (2) | <u><math>0.815 \pm 0.002</math></u> (2) | <b>1.6</b> |
|     |            |                                         |                                         |                                         |                                         |                                         |            |
| SVM | SAGE       | $0.758 \pm 0.011$ (4)                   | $0.847 \pm 0.008$ (3)                   | $0.777 \pm 0.015$ (3)                   | <b><math>0.695 \pm 0.004</math></b> (1) | <b><math>0.775 \pm 0.002</math></b> (1) | <u>2.4</u> |
|     | GAT        | <b><math>0.818 \pm 0.010</math></b> (1) | <b><math>0.871 \pm 0.005</math></b> (1) | <b><math>0.803 \pm 0.007</math></b> (1) | $0.373 \pm 0.004$ (4)                   | <u><math>0.629 \pm 0.004</math></u> (2) | <b>1.8</b> |
|     | GIN        | $0.788 \pm 0.007$ (3)                   | $0.776 \pm 0.008$ (4)                   | $0.775 \pm 0.013$ (4)                   | $0.370 \pm 0.004$ (3)                   | $0.625 \pm 0.012$ (3)                   | 3.4        |
|     | GCN (main) | <u><math>0.812 \pm 0.024</math></u> (2) | <u><math>0.859 \pm 0.004</math></u> (2) | <u><math>0.799 \pm 0.011</math></u> (2) | <u><math>0.411 \pm 0.005</math></u> (2) | <u><math>0.619 \pm 0.003</math></u> (4) | <u>2.4</u> |
|     |            |                                         |                                         |                                         |                                         |                                         |            |

graph; on Roman-Empire with SVM, feature-based baselines outperform both contrastive methods, highlighting that heterophilic settings remain challenging for contrastive approaches.

*Encoder ablation.* Table 3 reports results with GCN replaced by GAT, GraphSAGE, and GIN. GAT leads on homophilic datasets with the SVM classifier, where it ranks first on all three, but GCN is equally competitive on homophilic datasets with the MLP classifier. GraphSAGE ranks first on both heterophilic datasets across both classifiers. GCN is the most consistent encoder across all five datasets and pairs best with the MLP downstream classifier, which justifies its use as the primary encoder throughout the evaluation.

### 4.3 Limitations and Work in Progress

This paper is submitted as a work-in-progress contribution; the current experiments establish degree-preserving rewiring as a principled and competitive augmentation strategy, with a broader evaluation ongoing.

*Feature null model.* The configuration model randomises edge connectivity while leaving node features intact, providing no invariance signal over node attributes. A natural extension is a feature-space null model that preserves the marginal feature distribution while randomising attribute structure; we are currently integrating this with our edge rewiring.

*Task scope.* Our evaluation is limited to node classification; it remains to be seen whether the same benefits carry over to link prediction or graph classification, where the structurally relevant features differ. We are extending the evaluation to these tasks as part of the ongoing work.

*Evaluation breadth.* The comparison does not yet include recent structure-aware methods such as COSTA [35]. We are actively extending the benchmark to more precisely quantify where degree-preserving rewiring is most beneficial.

## 5 Conclusion

We presented degree-preserving edge rewiring as a principled augmentation strategy for graph contrastive learning, grounded in the configuration model from network science. Unlike random perturbations, which introduce degree-dependent noise and disrupt community structure, our MCMC edge-swap procedure generates views that preserve the exact degree sequence while randomising higher-order connectivity. This gives the contrastive objective a clear and interpretable invariance: the encoder is trained to be stable under degree-preserving rewiring, not under arbitrary structural noise.

Degree-preserving augmentation achieves lower InfoNCE training loss than random rewiring across all datasets and produces better-separated embeddings on homophilic graphs. In the edge-only setting, downstream gains are consistent on homophilic datasets while heterophilic datasets slightly favour random rewiring. In the full GraphCL pipeline, DP-Aug achieves the best average rank with the MLP classifier and is competitive to SVM.

The results suggest that structural constraints on augmentation are not merely theoretical. They measurably improve representation quality, and the degree sequence as the chosen invariant connects graph machine learning to an established null-model, with the configuration model as one instance of a broader class of constraint-preserving augmentations worth exploring.

**Acknowledgments.** J.P. and I.S. acknowledge funding by the German Ministry of Research, Technology and Space (BMFTR) under grant agreement No. 16IS24072E (COMFORT).

**Disclosure of Interests.** The authors have no competing interests to declare that are relevant to the content of this article.

## References

1. Bao, P., Yan, R., Pan, S.: Co-augmentation of structure and feature for boosting graph contrastive learning. *Information Sciences* **676**, 120792 (2024)
2. Casiraghi, G., Nanumyan, V.: Configuration models as an urn problem. *Scientific reports* **11**(1), 13416 (2021), <https://www.nature.com/articles/s41598-021-92519-y>
3. Chen, T., Kornblith, S., Norouzi, M., Hinton, G.: A simple framework for contrastive learning of visual representations. In: *International conference on machine learning*. pp. 1597–1607. PmLR (2020), <https://proceedings.mlr.press/v119/chen20j/chen20j.pdf>
4. Cortes, C., Vapnik, V.: Support-vector networks. *Machine learning* **20**(3), 273–297 (1995), <https://link.springer.com/article/10.1007/BF00994018>

5. Dwivedi, V.P., Joshi, C.K., Luu, A.T., Laurent, T., Bengio, Y., Bresson, X.: Benchmarking graph neural networks. *Journal of Machine Learning Research* **24**(43), 1–48 (2023), <https://jmlr.org/papers/volume24/22-0567/22-0567.pdf>
6. Fey, M., Lenssen, J.E.: Fast graph representation learning with pytorch geometric. *ArXiv abs/1903.02428* (2019), <https://api.semanticscholar.org/CorpusID:70349949>
7. Fosdick, B.K., Larremore, D.B., Nishimura, J., Ugander, J.: Configuring random graph models with fixed degree sequences. *Siam Review* **60**(2), 315–355 (2018), <https://epubs.siam.org/doi/10.1137/16M1087175>
8. Grover, A., Leskovec, J.: node2vec: Scalable feature learning for networks. In: *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. pp. 855–864 (2016), <https://dl.acm.org/doi/10.1145/2939672.2939754>
9. Hamilton, W., Ying, Z., Leskovec, J.: Inductive representation learning on large graphs. *Advances in neural information processing systems* **30** (2017), [https://papers.nips.cc/paper\\_files/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7e9bea9-Abstract.html](https://papers.nips.cc/paper_files/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7e9bea9-Abstract.html)
10. Hassani, K., Khasahmadi, A.H.: Contrastive multi-view representation learning on graphs. In: *Proceedings of the 37th International Conference on Machine Learning, ICML’20, JMLR.org* (2020)
11. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: Bengio, Y., LeCun, Y. (eds.) *ICLR (Poster)* (2015), <http://dblp.uni-trier.de/db/conf/iclr/iclr2015.html#KingmaB14>
12. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*. OpenReview.net (2017), <https://openreview.net/forum?id=SJU4ayYgl>
13. LaRock, T., Nanumyan, V., Scholtes, I., Casiraghi, G., Eliassi-Rad, T., Schweitzer, F.: Hypa: Efficient detection of path anomalies in time series data on networks. In: *Proceedings of the 2020 SIAM International Conference on Data Mining*. pp. 460–468. Society for Industrial and Applied Mathematics (Jan 2020). <https://doi.org/10.1137/1.9781611976236.52>, <http://dx.doi.org/10.1137/1.9781611976236.52>
14. Lim, D., Hohne, F., Li, X., Huang, S.L., Gupta, V., Bhalerao, O., Lim, S.N.: Large scale learning on non-homophilous graphs: New benchmarks and strong simple methods. In: Ranzato, M., Beygelzimer, A., Dauphin, Y.N., Liang, P., Vaughan, J.W. (eds.) *NeurIPS*. pp. 20887–20902 (2021), <http://dblp.uni-trier.de/db/conf/nips/neurips2021.html#LimHLHGBL21>
15. Maslov, S., Sneppen, K.: Specificity and stability in topology of protein networks. *Science* **296**(5569), 910–913 (2002), <https://www.science.org/doi/10.1126/science.1065103>
16. Molloy, M., Reed, B.: A critical point for random graphs with a given degree sequence. *Random structures & algorithms* **6**(2-3), 161–180 (1995), <https://onlinelibrary.wiley.com/doi/10.1002/rsa.3240060204>
17. Newman, M.E.: The structure and function of complex networks. *SIAM review* **45**(2), 167–256 (2003), <https://epubs.siam.org/doi/10.1137/S003614450342480?cookieSet=1>
18. Newman, M.E., Girvan, M.: Finding and evaluating community structure in networks. *Physical review E* **69**(2), 026113 (2004), <https://journals.aps.org/pre/abstract/10.1103/PhysRevE.69.026113>

19. Newman, M.E., Strogatz, S.H., Watts, D.J.: Random graphs with arbitrary degree distributions and their applications. *Physical review E* **64**(2), 026118 (2001), <https://journals.aps.org/pre/abstract/10.1103/PhysRevE.64.026118>
20. Oord, A.v.d., Li, Y., Vinyals, O.: Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018), <https://arxiv.org/abs/1807.03748>
21. von Pichowski, J., Perri, V., Qarkaxhija, L., Scholtes, I.: Inference of sequential patterns for neural message passing in temporal graphs (2024), <https://arxiv.org/abs/2406.16552>
22. Shchur, O., Mumme, M., Bojchevski, A., Günnemann, S.: Pitfalls of graph neural network evaluation. *CoRR* **abs/1811.05868** (2018), <http://dblp.uni-trier.de/db/journals/corr/corr1811.html#abs-1811-05868>
23. Suresh, S., Li, P., Hao, C., Neville, J.: Adversarial graph augmentation to improve graph contrastive learning. *Advances in Neural Information Processing Systems* **34**, 15920–15933 (2021), <https://papers.neurips.cc/paper/2021/file/854f1fb6f65734d9e49f708d6cd84ad6-Paper.pdf>
24. Thakoor, S., Tallec, C., Azar, M.G., Azabou, M., Dyer, E.L., Munos, R., Veličković, P., Valko, M.: Large-scale representation learning on graphs via bootstrapping. *International Conference on Learning Representations (ICLR)* (2022)
25. Tian, Y., Sun, C., Poole, B., Krishnan, D., Schmid, C., Isola, P.: What makes for good views for contrastive learning? *Advances in neural information processing systems* **33**, 6827–6839 (2020), <https://proceedings.neurips.cc/paper/2020/file/4c2e5eaae9152079b9e95845750bb9ab-Paper.pdf>
26. Veličković, P., Fedus, W., Hamilton, W.L., Liò, P., Bengio, Y., Hjelm, R.D.: Deep graph infomax. In: *ICLR (Poster)*. OpenReview.net (2019), <http://dblp.uni-trier.de/db/conf/iclr/iclr2019.html#VeličkovićFHLBH19>
27. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y.: Graph attention networks. In: *International Conference on Learning Representations* (2018), <https://openreview.net/forum?id=rJXmpikCZ>
28. Wei, X., Zhang, B.: A generative graph contrastive learning model with global signal. *arXiv preprint arXiv:2504.18148* (2025), <https://arxiv.org/abs/2504.18148>
29. Xia, J., Wu, L., Chen, J., Hu, B., Li, S.Z.: Simgrace: A simple framework for graph contrastive learning without data augmentation. In: *Proceedings of the ACM Web Conference 2022*. pp. 1070–1079. WWW '22, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3485447.3512156>, <https://doi.org/10.1145/3485447.3512156>
30. Xu, K., Hu, W., Leskovec, J., Jegelka, S.: How powerful are graph neural networks? (2018), <https://cs.stanford.edu/people/jure/pubs/gin-iclr19.pdf>
31. Yang, Z., Cohen, W.W., Salakhutdinov, R.: Revisiting semi-supervised learning with graph embeddings. In: *Proceedings of the 33rd International Conference on International Conference on Machine Learning - Volume 48*. pp. 40–48. ICML'16, JMLR.org (2016)
32. You, Y., Chen, T., Shen, Y., Wang, Z.: Graph contrastive learning automated. In: *International conference on machine learning*. pp. 12121–12132. PMLR (2021), <https://proceedings.mlr.press/v139/you21a/you21a.pdf>
33. You, Y., Chen, T., Sui, Y., Chen, T., Wang, Z., Shen, Y.: Graph contrastive learning with augmentations. *Advances in neural information processing systems* **33**, 5812–5823 (2020), <https://proceedings.nips.cc/paper/2020/file/3fe230348e9a12c13120749e3f9fa4cd-Paper.pdf>

34. Zhang, J., Luo, Y.: Degree centrality, betweenness centrality, and closeness centrality in social network. In: 2017 2nd international conference on modelling, simulation and applied mathematics (MSAM2017). pp. 300–303. Atlantis press (2017), <https://www.atlantis-press.com/proceedings/msam-17/25874733>
35. Zhang, Y., Zhu, H., Song, Z., Koniusz, P., King, I.: Costa: Covariance-preserving feature augmentation for graph contrastive learning. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 2524–2534. KDD '22, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3534678.3539425>, <https://doi.org/10.1145/3534678.3539425>
36. Zhu, Y., Xu, Y., Yu, F., Liu, Q., Wu, S., Wang, L.: Deep graph contrastive representation learning (2020), <https://arxiv.org/abs/2006.04131>
37. Zhu, Y., Xu, Y., Yu, F., Liu, Q., Wu, S., Wang, L.: Graph contrastive learning with adaptive augmentation. In: Proceedings of the web conference 2021. pp. 2069–2080 (2021), [https://link.springer.com/chapter/10.1007/978-3-031-26387-3\\_36](https://link.springer.com/chapter/10.1007/978-3-031-26387-3_36)

## A Degree-Preserving Edge Rewiring (DP-Rewire)

**Input:** Graph  $G = (V, E)$ , number of swap attempts  $k$   
**Output:** Rewired graph  $G' = (V, E')$

```

 $E' \leftarrow E;$ 
// Markov Chain Monte Carlo edge-swap sampling
for  $i = 1$  to  $k$  do
    Sample two distinct edges  $(u_1, v_1), (u_2, v_2) \in E'$  uniformly at random;
    // Proposed new edges
     $(u_1, v_2), (u_2, v_1);$ 
    // Check validity constraints
    if  $u_1 \neq v_2$  and  $u_2 \neq v_1$  and  $(u_1, v_2) \notin E'$  and  $(u_2, v_1) \notin E'$  and
         $|\{u_1, v_1, u_2, v_2\}| = 4$  then
        // Accept swap
         $E' \leftarrow E' \setminus \{(u_1, v_1), (u_2, v_2)\};$ 
         $E' \leftarrow E' \cup \{(u_1, v_2), (u_2, v_1)\};$ 
    end
end
return  $G' = (V, E');$ 

```

**Algorithm 1:** Degree-Preserving Edge Rewiring (DP-Rewire)

## B Datasets

Table 4: Overview of the datasets used in this work. Homophily is the class-insensitive edge homophily ratio [14].

| Dataset          | Nodes  | Edges   | Features | Classes | Avg. Degree | Homophily |
|------------------|--------|---------|----------|---------|-------------|-----------|
| Cora             | 2,708  | 10,556  | 1,433    | 7       | 3.90        | 0.7657    |
| PubMed           | 19,717 | 88,648  | 500      | 3       | 4.50        | 0.6641    |
| Amazon Computers | 13,752 | 491,722 | 767      | 10      | 35.76       | 0.7002    |
| Roman-Empire     | 22,662 | 32,927  | 300      | 18      | 2.91        | 0.0208    |
| Minesweeper      | 10,000 | 39,402  | 7        | 2       | 7.88        | 0.0094    |

## C Training Parameters

We train all GNN encoders using the Adam optimizer [11] with a learning rate of  $\eta = 0.001$  and a weight decay of  $\lambda = 10^{-5}$ . The contrastive temperature is set to  $\tau = 0.4$ , and training runs for 100 epochs. Each encoder has a hidden

dimension of  $d_h = 256$  and produces node embeddings with dimension  $d = 128$ . The number of edge swaps for the degree-preserving augmentation varies across datasets:  $k = 200,000$  for Amazon Computers,  $k = 5,000$  for Cora,  $k = 40,000$  for PubMed,  $k = 35,000$  for Roman-Empire and Minesweeper. We adjusted this parameter because of the different graph sizes. For Cora, PubMed, Roman-Empire, and Minesweeper, we use the public train, validation, and test splits provided by PyTorch Geometric [6]. For Amazon Computers, we randomly split nodes into training (80%), validation (10%), and test (10%) sets. Because contrastive pretraining and MLP optimization are stochastic, we repeat all experiments five times independently. We report the mean accuracy and standard deviation across runs to make comparisons between methods more reliable. The evaluation uses a standard two-stage self-supervised learning protocol [3]. First, the GNN encoder is pretrained with the contrastive objective and does not use node labels. Then, the encoder weights are frozen, and a downstream classifier is trained on labeled training nodes using only the frozen embeddings. This setup ensures that the reported accuracy reflects only the quality of the self-supervised representations, since the encoder is not updated with label information during evaluation. We evaluate two downstream classifiers. The first is a three-layer MLP with two hidden layers, batch normalization, ReLU activation, and dropout. It is trained with the Adam optimizer [11] and cross-entropy loss. We use grid search to tune hidden dimensions, dropout rates, learning rates, and weight decay and select the best model based on validation accuracy. The second classifier is a linear Support Vector Machine (SVM) [4] with balanced class weights. The regularization parameter  $C$  is chosen in a sweep based on validation accuracy. We adjust the augmentation parameters for each dataset; for Cora, we set the feature masking rate to  $p_f = 0.3$ , the edge drop rate to  $p_e = 0.2$ , and add Gaussian noise with  $\sigma = 0.05$ . For Amazon Computers, we use  $p_f = 0.3$ ,  $p_e = 0.1$ , and do not add feature noise ( $\sigma = 0.0$ ). We chose these values based on validation performance across different configurations.

### C.1 Baseline Methods

We compare our contrastive learning framework against three baselines. Each one is evaluated using the same downstream protocol as our approach. Node2Vec [8] embeddings are computed using random walks on the graph. We set the return parameter to  $p = 1.0$  and the in-out parameter to  $q = 0.5$  to encourage a breadth-first search. All Node2Vec embeddings have a dimension of  $d = 64$ . For our second baseline, we use Laplacian positional encodings [5]. We do this by adding the 32 smallest non-trivial eigenvectors of the normalized graph Laplacian to the current node features. This approach adds spectral structural information to the node representations without using any learned parameters. For the third baseline, we use raw node features as embeddings and feed them directly to the classifier, without any graph-based learning. This baseline measures how much of the classification performance is due to node features alone, without considering the graph structure.